Spring Boot & Microservices Master Repository

Welcome to the development repository containing a collection of hands-on Spring Boot modules and cloud-native microservice projects. This document serves as a comprehensive developer guide detailing the implementation steps, `pom.xml` dependencies, configurations, and core annotations used to build the functionality in each module.

Project Structure

The codebase is organized into isolated Maven projects located in the June directory, alongside centralized configuration files at the repository root.

- June/module1homework: Spring IoC & Dependency Injection.
- June/module2.homework: Spring MVC request validation and custom constraint validators.
- June/module3.homework: JPA entities mapping and bidirectional relationship handling.
- June/module4.homework: External HTTP clients (`RestClient`) and JPA lifecycle auditing.
- June/module5.homework: JWT authentication, custom security filters, and Spring Security.
- June/module7.homework: Multi-layer testing (Unit, MVC slice, JPA slice) and code coverage reporting.
- June/module12.homework: Advanced microservices architecture (Eureka, Config Server, Gateway, OpenFeign, Resilience4j, Kafka, Zipkin).
- Repository Root Configs: Git-backed configurations (`application.yml`, `module12.order.yml`, etc.) parsed by the Cloud Config Server.

Detailed Chapter Guides

Module 1: Spring IoC Container & Dependency Injection

Goal: Understand bean management, dependency injection models (constructor-based injection), and bean resolution techniques.

1. Dependencies Added ([pom.xml](file:///Users/macbook/Documents/GitHub/module12.config/June/module1homework

Uses the standard starter dependency for core Spring Boot context:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
</dependency>
```

2. Key Annotations Used

- `@SpringBootApplication`: Marks the entry point class Module1homeworkApplication.java. It triggers `@Configuration`, `@EnableAutoConfiguration`, and `@ComponentScan`.
- `@Component`: Indicated on classes like CakeBaker.java and ChocolateFrosting.java to make them candidate beans for Spring's Application Context.

- `@Qualifier("beanName")`: Used in the CakeBaker.java constructor to resolve ambiguity when injecting interfaces with multiple implementations (`Frosting` and `Syrup` implementations).

3. Execution Flow

1. Spring Boot bootstraps, scans the packages, and registers beans.
2. Multiple implementations of `Frosting` (`ChocolateFrosting`, `StrawberryFrosting`) are detected.
3. Spring resolves injection into the `CakeBaker` constructor using `@Qualifier` values.
4. Main application fetches `CakeBaker.class` from the `ApplicationContext` and triggers `bakeCake()`.

Module 2: Request Validation & Custom Constraint Validators

Goal: Implement robust, custom constraint validation on incoming REST payloads and globally handle validation exceptions.

1. Dependencies Added ([pom.xml](file:///Users/macbook/Documents/GitHub/module12.config/June/module2.homewor

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

2. Implementations & Annotations

- Standard Validators: Applied on fields in EmployeeEntity.java (e.g., `@NotNull`, `@NotBlank`, `@Min(18)`, `@Max(50)`, `@Email`, `@Past`, `@Future`, `@CreditCardNumber`, `@URL`).
- Custom Annotation Development:
  - Created `@interface` Password.java and Prime.java.
  - Used `@Constraint(validatedBy = PasswordValidator.class)` to bind the custom annotation to the validator logic.
- Custom Validators:
  - PasswordValidator.java implements `ConstraintValidator<Password, String>`. It validates string lengths and regex criteria (uppercase, lowercase, and special characters).
  - PrimeValidator.java implements `ConstraintValidator<Prime, Integer>`, executing a mathematical prime-number check.
- Controller-Level Triggering:
  - Annotated `@Valid @RequestBody` in DepartmentController.java to validate input constraints prior to invoking method logic.
- Centralized Exception Handling:
  - Decorated GlobalExceptionHandler.java with `@ControllerAdvice` and `@ExceptionHandler(Exception.class)` to intercept validation errors and convert them into structured HTTP response payloads.

Module 3: JPA Entities, Mappings & Relationships

Goal: Design database models with JPA/Hibernate, configure bidirectional mappings, and prevent serialization infinite recursion.

1. Dependencies Added ([pom.xml](file:///Users/macbook/Documents/GitHub/module12.config/June/module3.homewor

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
```

2. Key Annotations Used

- `@Entity` & `@Table(name = "table_name")`: Declares standard JPA-managed persistent entities.
- `@Id` & `@GeneratedValue(strategy = GenerationType.IDENTITY)`: Configures database auto-increment primary keys.
- Mappings:
  - `@OneToOne`: Configured in Student.java mapping `AdmissionRecord` with cascade features (`cascade = CascadeType.ALL`).
  - `@ManyToOne` & `@JoinColumn(name = "author_id")`: Configures relationship mapping in Book.java targeting `Author`.
  - `@ManyToMany` & `@JoinTable`: Configured in Student.java and Subject.java specifying join tables (`student_professor`, `subject_student`), join columns, and inverse join columns.
- Recursion Control:
  - `@JsonIgnoreProperties`: Applied to relationships (e.g. `@JsonIgnoreProperties("students")`) to break cyclic references during Jackson JSON serialization.

Module 4: External API Integration & JPA Auditing

Goal: Consume external APIs using Spring 6 `RestClient`, log operation stages, and audit entity creations/updates automatically.

1. Dependencies Added ([pom.xml](file:///Users/macbook/Documents/GitHub/module12.config/June/module4.homewor

- Uses web starters for REST clients and Data JPA for auditing.

2. Key Implementations

- `RestClient` Configuration:
  - Registered a central `RestClient` bean in CurrencyConfig.java using `RestClient.builder().build()`.
- API Call & Caching:
  - Used `RestClient` in CurrencyService.java to request rates from `https://api.freecurrencyapi.com/v1/latest`.
  - Processed JSON mapping via `Map.class` to retrieve properties, calculate conversions, and cache transactions

into the local PostgreSQL database.

- JPA Auditing Setup:
  - Enabled auditing in the main class Application.java via `@EnableJpaAuditing`.
  - Created a base class Audit.java annotated with `@MappedSuperclass` and `@EntityListeners(AuditingEntityListener.class)`.
  - Marked auditing properties using `@CreatedDate` and `@LastModifiedDate` to auto-populate timestamps.
  - Entities like Currency.java inherit from `Audit` to inherit auditing attributes.

Module 5: Spring Security & JWT Authentication

Goal: Build a stateless security architecture using JWT access and refresh tokens, secure routes, and execute custom request interceptor filters.

1. Dependencies Added ([pom.xml](file:///Users/macbook/Documents/GitHub/module12.config/June/module5.homewor

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
```

2. Key Configurations & Code

- Security Configuration:
  - In SecurityConfig.java, disabled CSRF protection, permitted public authentication endpoints (`/api/v1/auth/`), and enforced authorization requirements (`.anyRequest().authenticated()`) on other routes.
- Custom Authentication Filters:
  - Created LoggingFilter.java extending `OncePerRequestFilter`.
  - Captured HTTP methods, request URIs, response states, and client execution times before forwarding the exchange chain.
  - Registered the filter in `SecurityFilterChain` prior to `UsernamePasswordAuthenticationFilter` via `.addFilterBefore()`.
- JWT Utilities:

- Created JwtService.java utilizing Java JWT (jjwt) libraries to generate access tokens (15-min duration) and refresh tokens (7-day duration), sign payloads with HMAC-SHA keys (`SignatureAlgorithm.HS256`), extract claims, and evaluate expiration states.

Module 7: Testing Frameworks, Mocks & Code Coverage

Goal: Design unit tests, web slice tests, database slice tests, mock external calls, and run test coverage audits.

1. Dependencies Added ([pom.xml](file:///Users/macbook/Documents/GitHub/module12.config/June/module7.homewor

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa-test</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
</dependency>
```

Added JaCoCo build plugin to count execution metrics and output HTML reports:

```
<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.11</version>
    <executions>
        <execution>
            <goals>
                <goal>prepare-agent</goal>
            </goals>
        </execution>
        <execution>
            <id>report</id>
            <phase>test</phase>
            <goals>
                <goal>report</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

2. Key Testing Strategies

- Service Unit Tests:
  - Extended with BookServiceTest.java using `MockitoExtension.class`.
  - Used `@Mock` for repository dependencies and `@InjectMocks` to initialize services.
  - Captured dynamic parameters passed during calls using Mockito's `ArgumentCaptor` and verified execution frequencies using `verify()`.
- JPA Repository Slice Tests:
  - In AuthorRepositoryTest.java, used `@DataJpaTest` to load JPA components.
  - Set `@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)` to test against the active, real database setup instead of memory mocks.

- Controller Slice Tests:
  - Used `@WebMvcTest(BookController.class)` in BookControllerTest.java.
  - Injected `MockMvc` and registered Mockito mocks using `@MockitoBean` (introduced in Spring Boot 3.4 to replace `@MockBean`).
  - Dispatched mock payloads via `mockMvc.perform(post(...))` and asserted status codes and contents using `status().isOk()` and `content().string()`.

Module 12: Microservices Cloud Infrastructure & Resiliency

Goal: Design a complete, cloud-native microservices mesh featuring centralized configurations, dynamic discovery, load-balanced routing, Feign client proxies, Kafka events, resilient circuits, and tracing logs.

```
flowchart TD
    ConfigServer["Cloud Config Server (Port 8888)"]
    GitRepo["Git Repository Configs"]
    EurekaServer["Eureka Registry (Port 8761)"]
    Gateway["API Gateway (Port 8085)"]
    OrderService["Order Service (Port 8081)"]
    InventoryService["Inventory Service (Port 8082)"]
    ShippingService["Shipping Service (Port 8083)"]
    Kafka["Apache Kafka (Port 9092)"]
    Zipkin["Zipkin Server (Port 9411)"]

    GitRepo --> ConfigServer
    ConfigServer -.->|Provides Configs| Gateway & OrderService & InventoryService & ShippingService

    Gateway -.->|Registers| EurekaServer
    OrderService -.->|Registers| EurekaServer
    InventoryService -.->|Registers| EurekaServer
    ShippingService -.->|Registers| EurekaServer

    Gateway -->|Routes via lb://| OrderService
    OrderService -->|Feign / Synchronous| InventoryService
    OrderService -->|Feign + CB + Retry| ShippingService

    OrderService -->|Publish order_created| Kafka
    Kafka -->|Consume order_created| InventoryService
    InventoryService -->|Publish order_status_updated| Kafka
    Kafka -->|Consume order_status_updated| OrderService

    OrderService & InventoryService & Gateway -->|Send Spans| Zipkin
```

1. Microservice Components & Configs

A. Central Cloud Config Server (module13.config)

- Dependency: `spring-cloud-config-server`.
- Annotations: Enabled configuration server capabilities using `@EnableConfigServer` on the main application class ConfigApplication.java.
- YAML Config (application.yaml):

```
  spring:
```

```
    cloud:
      config:
        server:
          git:
            uri: https://github.com/mdirfancse2023/module12.config
            default-label: main
```

Forces services to read database configurations (application.yml), circuit breaker properties (module12.order.yml), and port numbers from Git storage.

B. Service Discovery Registry (module12.eureka)

- Dependency: `spring-cloud-starter-netflix-eureka-server`.
- Annotations: `@EnableEurekaServer` on the application entry class EurekaApplication.java.
- YAML Config (application.yaml):

Disabled self-registration with `register-with-eureka: false` and `fetch-registry: false` to keep the Registry clean.

C. API Gateway Router (module12.apigateway)

- Dependency: `spring-cloud-starter-gateway-server-webflux` and `spring-cloud-starter-loadbalancer`.
- YAML Config (module12.apigateway.yml):

Specifies paths and matches them to services registered in Eureka using `lb://` load-balancing formats:

```
  spring:
    cloud:
      gateway:
        routes:
          - id: order-route
            uri: lb://module12.order
            predicates:
              - Path=/orders/**
```

- Custom Global Filter:

Developed RoleAuthorizationFilter.java implementing `GlobalFilter`. Intercepts incoming Webflux HTTP requests, checks authorization, extracts JWT claims via `JwtUtil`, matches role credentials, and responds with `401 Unauthorized` or `403 Forbidden` if keys/roles are invalid.

D. Declarative Feign Clients (module12.order)

- Dependency: `spring-cloud-starter-openfeign`.
- Annotations: Enabled feign client generation via `@EnableFeignClients` on the application class OrderApplication.java. Created interfaces like InventoryClient.java annotated with `@FeignClient(name = "module12.inventory")` to trigger remote HTTP executions dynamically.

E. Resiliency (Circuit Breakers & Retries)

- Dependency: `spring-cloud-starter-circuitbreaker-resilience4j`.
- Annotations: Configured on service layers like ShippingIntegrationService.java:
  - `@CircuitBreaker(name = "module12.shipment", fallbackMethod = "shipmentFallback")`
  - `@Retry(name = "shippingRetry")`
- Config Properties (module12.order.yml):

```
  resilience4j:
    circuitbreaker:
```

```
    instances:
      shippingService:
        sliding-window-size: 10
        minimum-number-of-calls: 5
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
  retry:
    instances:
      shippingRetry:
        max-attempts: 3
        wait-duration: 2s
```

F. Event-Driven Messaging (Apache Kafka)

- Dependency: `spring-boot-starter-kafka`.

- Topic Configuration:

Implemented KafkaTopicConfig.java declaring `NewTopic` beans (`order_created` and `order_status_updated`) to auto-create topics on the brokers.

- Producer Integration:

Created OrderProducer.java using `KafkaTemplate<String, Object>` to publish serializable events asynchronously, handling result futures using `.whenComplete()`.

- Consumer Listeners:

Annotated service classes using `@KafkaListener(topics = "...", groupId = "...")` in OrderStatusConsumer.java and InventoryConsumer.java to dynamically fetch and process event payloads.

G. Distributed Tracing (Zipkin)

- Dependency: `spring-boot-starter-zipkin` and actuator.

- YAML Config (application.yml):

```
management:
  tracing:
    sampling:
      probability: 1.0
  zipkin:
    tracing:
      endpoint: http://localhost:9411/api/v2/spans
```

Exports trace spans from Gateway, Order, and Inventory services to Zipkin for end-to-end request tracking.

How to Run the Ecosystem Local Environment

1. Prerequisite Infrastructure

Ensure you have Docker containers running PostgreSQL, Apache Kafka (with Zookeeper), and Zipkin:

```
# Start PostgreSQL Database
docker run -d --name pg-vector -p 5434:5432 -e POSTGRES_PASSWORD=admin123 -e POSTGRES_DB=vectordb
pgvector/pgvector:latest
```

```
# Start Zipkin
docker run -d -p 9411:9411 openzipkin/zipkin

# Start Kafka
docker-compose up -d  # Make sure you have your kafka compose script running
```

2. Boot Service Startup Order

Start the services in the following sequence using your IDE or terminal:

1. Config Server: Start ConfigApplication (Port `8888`)
2. Eureka Registry: Start EurekaApplication (Port `8761`)
3. Microservices (Wait for config server registry registration):
    - Start InventoryApplication (Port `8082`)
    - Start ShippingApplication (Port `8083`)
    - Start OrderApplication (Port `8081`)
4. API Gateway Router: Start GatewayApplication (Port `8085`)